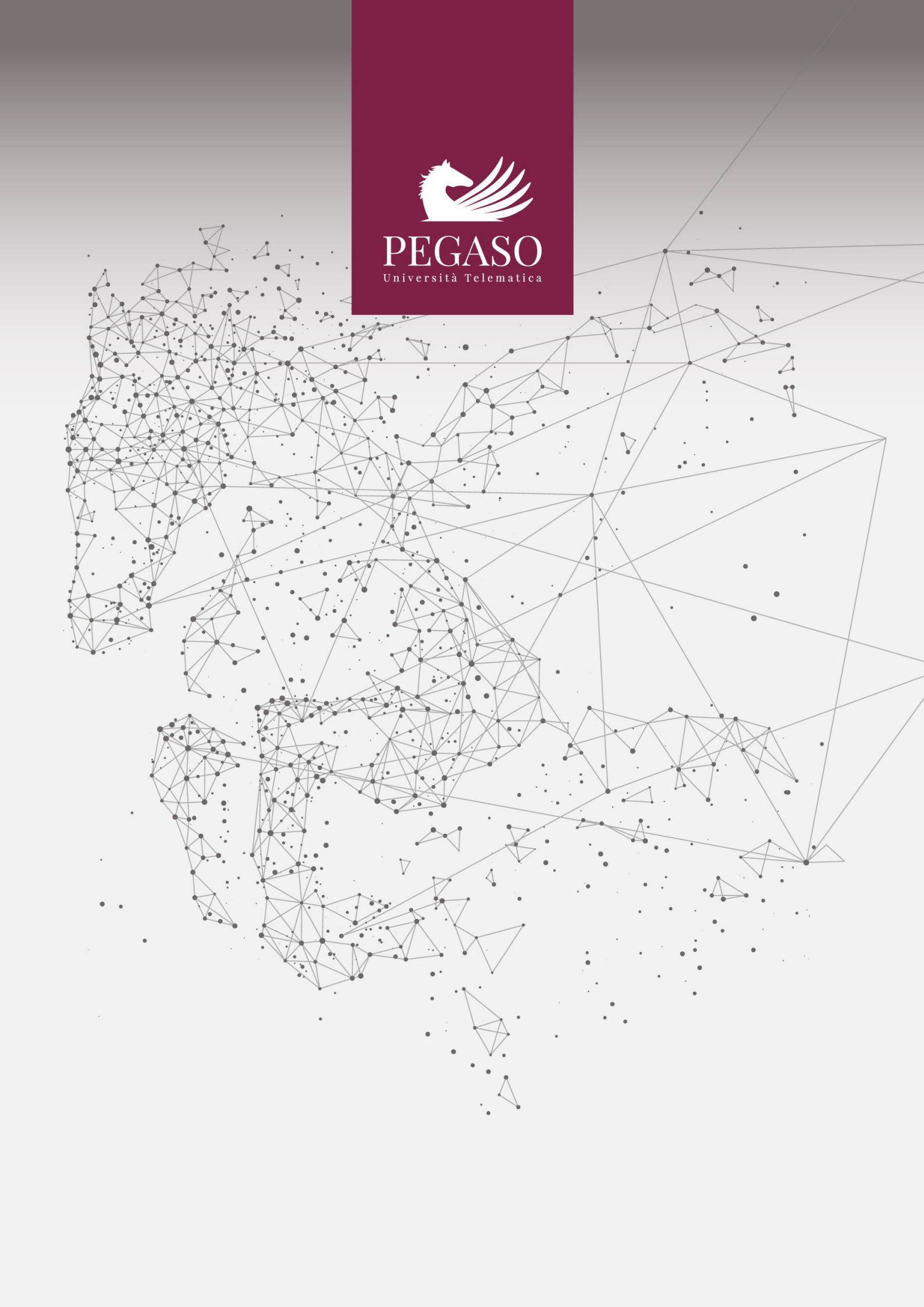

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. JAVASCRIPT 3
2. DOVE INSERIRE UN CODICE JAVASCRIPT 5
3. SINTASSI 8
BIBLIOGRAFIA E SITOGRAFIA 13

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Javascript

Il JavaScript è un linguaggio di programmazione che viene utilizzato principalmente per creare siti web interattivi e dinamici.

È stato creato negli anni '90 da Brendan Eich, uno sviluppatore di Netscape, ed è stato progettato per essere eseguito all'interno dei browser web.

Il JavaScript è un linguaggio di scripting interpretato, il che significa che non richiede di essere compilato e può essere eseguito direttamente nel browser. Questo rende il JavaScript molto flessibile e facile da utilizzare per creare funzionalità dinamiche e reattive sui siti web.

La differenza tra un linguaggio di programmazione compilato e uno interpretato è nel modo in cui viene eseguito il codice sorgente del programma:

- in un linguaggio di programmazione compilato, il codice sorgente viene prima tradotto in un linguaggio macchina, ovvero il codice binario che il computer può eseguire direttamente; il processo di compilazione traduce l'intero programma in un file eseguibile, che può essere poi eseguito dall'utente
- in un linguaggio interpretato, il codice sorgente viene eseguito direttamente, senza la necessità di una fase di compilazione; il codice sorgente viene passato ad un interprete, che esegue le istruzioni una per volta e produce l'output desiderato.

In altre parole, il processo di compilazione traduce l'intero programma in un file eseguibile che può essere poi eseguito in qualsiasi momento, mentre nell'interpretazione il codice sorgente viene eseguito immediatamente senza essere tradotto in un file eseguibile.

Questo rende il JavaScript molto flessibile, in quanto è possibile eseguire il codice JavaScript direttamente all'interno del browser e aggiornarlo in tempo reale senza dover ricompilare il codice. Tuttavia, poiché il codice viene eseguito in tempo reale, l'interprete JavaScript potrebbe essere meno efficiente di un programma compilato e richiedere maggiori risorse del sistema.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il JavaScript può essere utilizzato per una vasta gamma di attività, tra cui la validazione dei dati, la manipolazione del DOM (Document Object Model), l'animazione, la gestione degli eventi e la comunicazione con i server web attraverso le API.

Oltre alle funzionalità di base, il JavaScript è supportato da una vasta gamma di librerie e framework, come ad esempio React, Vue.js e AngularJS, che semplificano lo sviluppo di applicazioni web più complesse.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Dove inserire un codice Javascript

In una pagina HTML, il codice Javascript si inserisce all'interno dei tag <script> </script>.

In JavaScript, l'alert() è una funzione predefinita che viene utilizzata per visualizzare una finestra di dialogo di avviso con un messaggio all'interno.

La sintassi di base della funzione alert() è la seguente:

```
alert("Hello World");
```

Dove "Hello World" è il testo che si vuole visualizzare nella finestra di dialogo di avviso.

L'alert() viene utilizzato comunemente per fornire feedback all'utente o per avvisare l'utente di un'azione o di una condizione importante. Ad esempio, può essere utilizzato per confermare che un'azione è stata completata con successo, per richiedere all'utente di confermare l'eliminazione di un elemento, per segnalare un errore o per mostrare un messaggio personalizzato di benvenuto.

La finestra di dialogo di avviso prodotta dall'alert() è generalmente una finestra modale, il che significa che l'utente deve interagire con essa prima di poter continuare a utilizzare la pagina.

Se volessimo inserire tale alert all'interno della pagina HTML dovremmo dunque usare la seguente sintassi:

```
<script>alert("Hello World");</script>
```

Gli alert sono spesso utilizzati durante lo sviluppo e il debugging di applicazioni JavaScript; tuttavia, un altro strumento per aiutare a identificare problemi e monitorare il comportamento dell'applicazione è la console:

```
console.log("Hello World");
```

Dove "Hello World" è il testo che si desidera scrivere nella console. È possibile accedere alla console direttamente passando per gli "strumenti per sviluppatori" in Chrome, "esamina" in Edge o utilizzando strumenti analoghi in altri browser:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

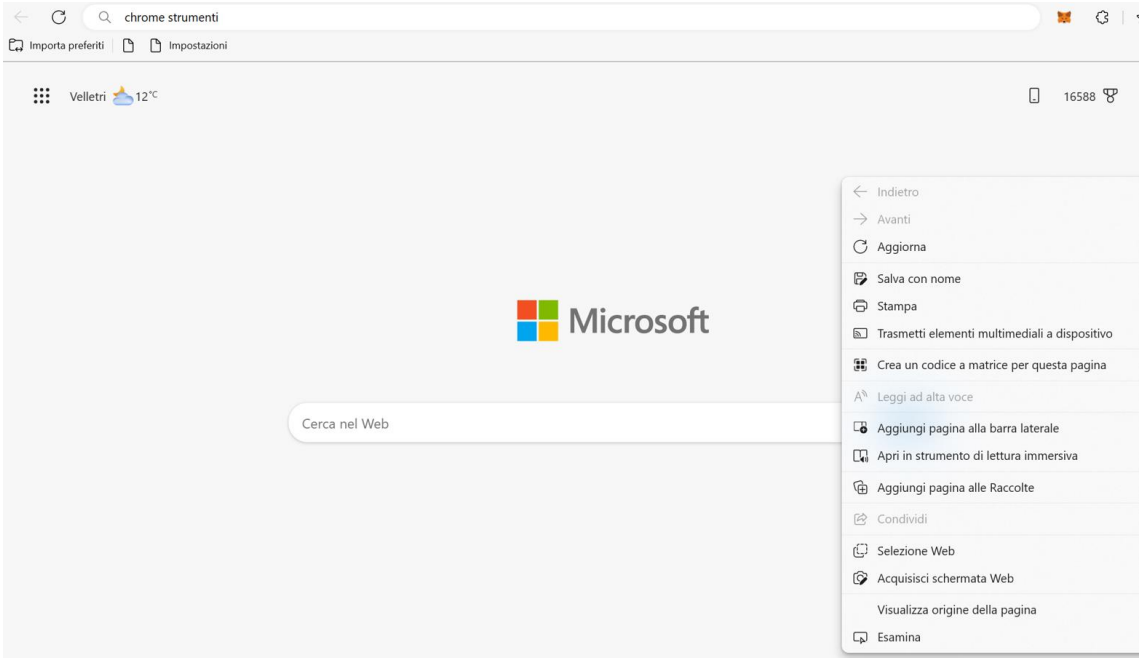


Figura 1 - Esamina in Edge

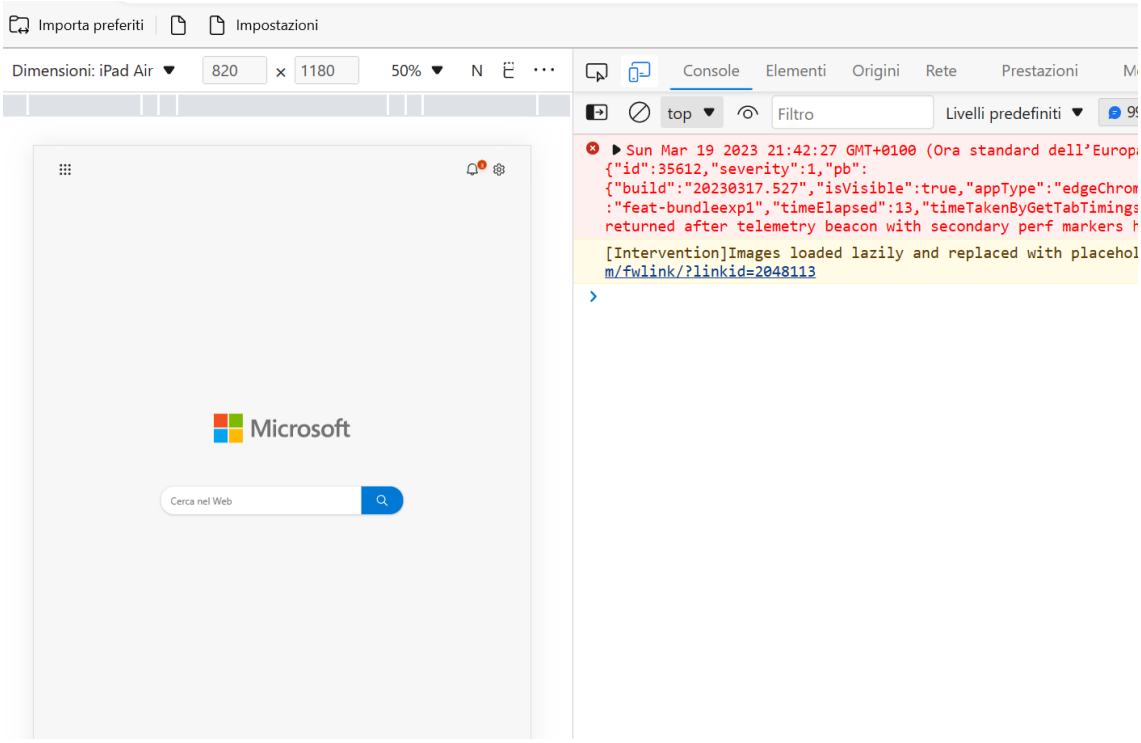


Figura 2 - Console in Edge

Tutto quello che verrà "loggato" sulla console, sarà visibile attraverso questo strumento.

Ma dove si deve posizionare un codice Javascript?

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Abbiamo 3 possibilità di inserire tale script nella pagina:

1. Nell'head (posizione 1)
2. Nel body subito dopo il tag body (posizione 2)
3. In un punto qualunque del body (posizione 3)

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>TEST</title>
  <!-- POSIZIONE 1 -->
</head>
<body>
  <!-- POSIZIONE 2 -->
  <p id="myP">Hello world</p>
  <!-- POSIZIONE 3 -->
</body>
</html>
```

1. Se lo script si trova nell'head, questo viene invocato immediatamente; se questo dipende da elementi specifici della pagina HTML lo script potrebbe non funzionare correttamente
2. Se lo script si trova nel body, questo viene invocato immediatamente durante il caricamento della pagina rispettando il suo posizionamento; se lo script dipende da elementi specifici della pagina HTML che si trovano DOPO (a livello di posizionamento) il caricamento dello script, lo script potrebbe non funzionare correttamente
3. Se lo script si trova nel body subito prima la chiusura del tag stesso, siamo certi che tutti gli elementi della pagina sono stati caricati, pertanto se lo script dipende da elementi specifici della pagina HTM, siamo certi che lo script funzionerà correttamente

È inoltre possibile scrivere il codice Javascript all'interno di un file esterno (tipicamente con estensione .js) e richiamarlo in una delle posizioni indicate in precedenza, attraverso la sintassi:

```
<script src="script.js"></script>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Sintassi

La sintassi JavaScript definisce due tipi di valori:

- Fissi (fixed)
- Variabili (variables)

I valori fissi sono chiamati letterali (literals) mentre quelli variabili sono chiamati variabili (variabiles). Le due regole di sintassi più importanti per i valori fissi sono:

- I numeri sono scritti con o senza decimali
- Le stringhe sono scritte tra singoli i doppi apici

In JavaScript, sia la parola chiave **var** che la parola chiave **let** vengono utilizzate per dichiarare variabili, ma hanno alcune differenze importanti. La principale differenza tra var e let è che le variabili dichiarate con var hanno un ambito di funzione (function scope), mentre le variabili dichiarate con let hanno un ambito di blocco (block scope). Altre peculiarità del let sono:

- le variabili definite con let non possono essere ridefinite
- le variabili definite con let devono essere dichiarate prima dell'utilizzo

Ad es. non è ammesso:

```
let x = "John Doe";
let x = 0;
```

ma è ammesso:

```
var x = "John Doe";
var x = 0;
```

ATTENZIONE: tutti gli identificatori in Javascript sono "case-sensitive" (si distingue cioè tra maiuscolo e minuscolo).

Tipicamente le variabili si definiscono mediante alcune delle seguenti convenzioni:

- Underscore: first_name, last_name …
- Upper Camel Case: FirstName, LastName …

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- Lower Camel Case: firstName, lastName …

JavaScript ha 8 tipi di dato:

1. String
2. Number: double (floating point a 64 bit)
3. Bigint: numeri interi più grandi di "Number"
4. Boolean
5. Undefined: una variabile senza valore
6. Null
7. Symbol
8. Object

I tipi di dato sono gestiti dinamicamente, questo significa che la stessa variabile può assumere tipi diversi:

```
let x;       // x è indefinito
x = 5;       // x è un numero
x = "John";  // x è una stringa
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Condizione

Le istruzioni condizionali vengono utilizzate per eseguire azioni diverse in base a diverse condizioni.

In JavaScript abbiamo le seguenti istruzioni condizionali:

- Usare **if** per specificare un blocco di codice da eseguire, se una condizione specificata è vera
- Usare **else** per specificare un blocco di codice da eseguire, se la stessa condizione è falsa
- Usare **else if** per specificare una nuova condizione da testare, se la prima condizione è falsa
- Usare **switch** per specificare molti blocchi alternativi di codice da eseguire

```
if (time < 12) {
  saluto = "Buongiorno";
} else if (time < 18) {
  saluto = "Buon pomeriggio";
} else {
  saluto = "Buonasera";
}
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Loops

JavaScript supporta diversi tipi di cicli:

- **for** - cicla attraverso un blocco di codice un numero di volte
- **for/in** - cicla attraverso le proprietà di un oggetto
- **for/of** - cicla attraverso i valori di un oggetto iterabile
- **while** - cicla attraverso un blocco di codice finché una specifica condizione è vera
- **do/while** - cicla anche attraverso un blocco di codice finché una specifica condizione è vera

Lo statement **for** crea un ciclo con 3 espressioni opzionali:

```
for (espressione1; espressione2; espressione3) {
// blocco di codice da eseguire
}
```

- L'espressione1 viene eseguita (una sola volta) prima dell'esecuzione del blocco di codice.
- L'espressione2 definisce la condizione per l'esecuzione del blocco di codice.
- L'espressione3 viene eseguita (ogni volta) dopo l'esecuzione del blocco di codice.

Di solito si utilizza l'espressione 1 per inizializzare la variabile utilizzata nel ciclo (let i = 0). Tale espressione è comunque facoltativa. È inoltre possibile inizializzare diversi valori nell'espressione 1 (separati da virgola).

Spesso l'espressione 2 viene utilizzata per valutare la condizione della variabile iniziale. Tale espressione è comunque facoltativa. Se l'espressione 2 restituisce true, il ciclo inizierà di nuovo. Se restituisce false, il ciclo terminerà. Se si omette l'espressione 2, è necessario fornire un break all'interno del ciclo, altrimenti il ciclo non terminerà mai e questo farà crashare il browser.

L'espressione 3 è tipicamente usata per incrementare il valore della variabile iniziale. Tale espressione è comunque facoltativa. L'espressione 3 può fare qualsiasi cosa, come un decremento negativo (i--), un incremento positivo (i = i + 15), o qualsiasi altra cosa. L'espressione 3 può anche essere omessa (come quando si incrementano i valori all'interno del ciclo).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il ciclo **while** esegue un blocco di codice fintanto che una determinata condizione risulta essere vera. La sintassi del ciclo while è la seguente:

```
while (condizione) {
  // blocco di codice da eseguire
}
```

La condizione è un'espressione che viene valutata all'inizio di ogni iterazione del ciclo. Se la condizione è vera, il blocco di codice all'interno del ciclo viene eseguito. Dopo l'esecuzione del blocco di codice, la condizione viene valutata nuovamente. Questo processo continua fino a quando la condizione diventa falsa, momento in cui il ciclo while termina e l'esecuzione del programma continua al di fuori del ciclo.

Il ciclo **do...while** è simile al ciclo while, ma ha una differenza significativa: il blocco di codice all'interno del ciclo viene eseguito almeno una volta, indipendentemente dalla verità della condizione. La sintassi del ciclo do...while è la seguente:

```
do {
  // blocco di codice da eseguire
} while (condizione);
```

Il blocco di codice all'interno del ciclo viene eseguito una volta senza verificare la condizione. Dopo l'esecuzione del blocco di codice, la condizione viene valutata. Se la condizione è vera, il blocco di codice viene eseguito di nuovo e il processo continua finché la condizione diventa falsa.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia e sitografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005
- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).